Software Requirements Specification

Doctor_LLM / MedBot

Version 1.0 (Draft) | 17 August 2026

Document purpose

This specification defines the functional, non-functional, security, and technical requirements for Doctor_LLM / MedBot. The system is a web-based medical-information assistant built around Retrieval-Augmented Generation (RAG).

Safety notice: The system provides educational information only. It must not present itself as a doctor, diagnose a condition, prescribe treatment, or replace urgent medical care.

1. Project overview

Doctor_LLM / MedBot answers questions using approved medical documents. It retrieves relevant passages from Pinecone and uses an OpenAI language model to produce a grounded answer. Supabase supports user and application data; Flask exposes the backend API.

2. Scope

- User registration, login, and JWT-based authentication.
- Upload and ingestion of PDF medical documents.
- PDF text extraction, chunking, embedding, and Pinecone indexing.
- Authenticated question answering using retrieved document context.
- Source-aware responses with a medical safety disclaimer.
- Persistent user/application data through Supabase.
- Structured console and file logging.

Out of scope for version 1

- Diagnosis, prescriptions, emergency triage, or access to hospital records.
- OCR extraction from scanned PDFs.
- Guaranteeing support for languages not supported by the selected model or source documents.

3. Users and roles

Role	Description	Main permissions
Visitor	Unauthenticated user	Register and log in
User	Authenticated end user	Ask questions and view their own history
Administrator	Project operator	Upload/manage approved documents and monitor logs

4. Functional requirements

FR-1: Configuration and startup

1. The backend shall load configuration from backend/.env at startup.
2. The backend shall stop with a clear error when a required configuration value is missing.
3. The backend shall create logs/ and data/ folders if they do not exist.
4. Secrets shall not be written to source code or logs.

Required values include OpenAI API key, Pinecone API key, Supabase URL, Supabase service key, Flask secret key, and JWT secret.

FR-2: User authentication

1. A visitor shall be able to register with valid credentials.
2. A registered user shall be able to log in.

3. The system shall issue a signed JWT on successful login.
4. Protected endpoints shall reject missing, invalid, expired, or tampered JWTs with HTTP 401.
5. JWT expiry shall be configurable through JWT_EXPIRY_HOURS.
6. Passwords shall be stored only as bcrypt hashes.

FR-3: Document ingestion

1. An administrator shall be able to submit a PDF for ingestion.
2. The system shall validate file type and reject non-PDF uploads.
3. The system shall extract text from supported PDFs using PyPDF.
4. The system shall split text using configurable CHUNK_SIZE and CHUNK_OVERLAP values.
5. The system shall create embeddings using the configured OpenAI embedding model.
6. The system shall store vectors and metadata in the configured Pinecone index.
7. The system shall report ingestion status and processed chunk count.

FR-4: Medical question answering

1. An authenticated user shall be able to submit a medical-information question.
2. The system shall embed the question and retrieve the top RETRIEVER_TOP_K relevant chunks from Pinecone.
3. The prompt shall instruct the model to use retrieved context and state uncertainty when context is insufficient.
4. The system shall use the configured OpenAI model, temperature, and maximum token limit.
5. Every response shall include a short disclaimer that it is not professional medical advice.
6. The system shall advise users to seek professional or emergency help for possible emergencies.
7. The system shall return source metadata when available.

FR-5: Conversation history

1. The system shall store the authenticated user's questions, answers, timestamps, and available source metadata in Supabase.
2. A user shall be able to retrieve only their own history.
3. Administrative access to user data shall be restricted and auditable.

FR-6: Logging and errors

1. The backend shall log events to the console and to logs/medbot.log.
2. The logging level shall be controlled by LOG_LEVEL.
3. The logging system shall avoid duplicate handlers and duplicate messages.
4. Client error responses shall not expose secrets, stack traces, or internal database details.

5. Proposed API requirements

Method	Endpoint	Auth	Purpose
POST	/api/auth/register	No	Create user account
POST	/api/auth/login	No	Authenticate and return JWT
POST	/api/documents/upload	Admin	Upload and ingest a PDF
POST	/api/chat	User	Ask a question and receive RAG response
GET	/api/chat/history	User	Retrieve current user's history
GET	/api/health	No	Service health check

POST /api/chat request

```
{ "question": "What are common symptoms of diabetes?" }
```

Response

```
{ "answer": "...", "sources": [{ "document": "diabetes-guide.pdf", "page": 4 }], "disclaimer": "This is general medical information, not medical advice." }
```

6. Non-functional requirements

Security

1. .env shall be included in .gitignore and never committed.
2. HTTPS shall be used in deployed environments.
3. Supabase service-role keys shall remain backend-only.
4. Inputs, uploads, and JWTs shall be validated.
5. Rate limiting should protect authentication and chat endpoints.

Performance

1. The health endpoint should respond within 1 second under normal conditions.
2. A normal chat request should target a response within 10 seconds, excluding upstream provider outages.
3. Document processing may be asynchronous for large PDFs.

Reliability and maintainability

1. The application shall log failed API calls, ingestion errors, and authentication failures.
2. Failure of OpenAI, Pinecone, or Supabase shall produce a safe error response and no data corruption.
3. Configuration shall be centralized in config.py and logging obtained through get_logger(__name__).
4. Unit tests shall cover configuration, authentication, ingestion validation, and basic API behaviour.

7. Technology constraints

Area	Selected technology
Backend API	Flask and Flask-CORS
LLM and embeddings	OpenAI / LangChain OpenAI
RAG orchestration	LangChain
Vector database	Pinecone
PDF parsing	PyPDF
User/database platform	Supabase
Authentication	PyJWT and bcrypt
Configuration	python-dotenv
Logging	Python logging and colorlog

8. Acceptance criteria

- The backend starts successfully only with valid required environment variables.
- User registration and login return a valid JWT.
- A protected API rejects a request without a valid JWT.
- A valid PDF is chunked and its vectors are added to the configured Pinecone index.
- A question returns a context-based answer, a disclaimer, and source information when available.
- The app creates and writes to logs/medbot.log without duplicate entries.
- No API key, service key, password, or secret appears in Git or normal log output.

9. Risks and decisions needed

- Confirm the Pinecone index type/region and ensure its dimension is 1536 for text-embedding-3-small.
- Define administrator authorization before exposing document upload endpoints.
- Define document approval and update procedures to reduce unsafe or outdated medical content.
- Define retention/deletion policy for chat history and uploaded documents.
- Add a clear safety escalation policy for emergency symptoms.